InstantNGP vs 3D Gaussian Splatting

E9 247 : Assignment 2

Name : Soham Gaonkar

SR no.: 28875

This report compares InstantNGP [1] and 3D Gaussian Splatting (3DGS) [2] on a self-captured static indoor scene, using the same images, camera poses, train/test split, resolution, GPU and metric code for both methods.

1. Dataset

The scene is a wall shelf with books, plants, trophies and a few ornaments. It was recorded on a phone as a 31 s portrait video (1080 x 1920, 30 fps), walking sideways along the shelf at the same height and without turning the camera. Frames were extracted at 6 fps (186 frames), and the sharpest of every 3 consecutive frames was kept using the variance of the Laplacian. This left 62 frames, and none had to be dropped after checking them by eye.

COLMAP [3] registered all 62 images (16,573 points, 0.40 px reprojection error) and undistorted them to 701 x 1249. Every 8th image was held out for testing, the same split 3DGS uses with --eval, which gives 54 training and 8 test images.

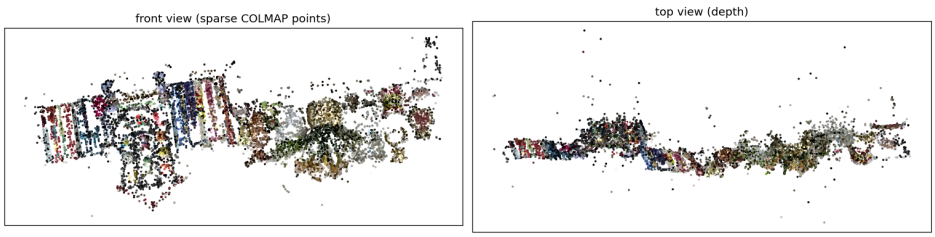


Figure 1. COLMAP sparse point cloud (front and top view).

2. Methods and settings

	3DGS	**InstantNGP**
Code	graphdeco-inria/gaussian-splatting, commit `54c035f`	NVlabs/instant-ngp, commit `abe236e`
Model	3D Gaussians initialised from the 16,573 COLMAP points, SH degree 3	hash grid (8 levels x 4 features, table size 2^{19}), density MLP 1 x 64, colour MLP 2 x 64
Training	30,000 iterations (default), loss 0.8·L1 + 0.2·D-SSIM	35,000 steps (default), continued to 100,000 to check convergence
Densification / bounds	every 100 iterations from 500 to 15k: clone/split high-gradient Gaussians, prune opacity < 0.005; opacity reset every 3k	`aabb_scale` 4
Checkpoints	1k, 3k, 7k, 15k, 30k	1k, 3k, 7k, 15k, 35k, 100k
Evaluation renders	full resolution, 701 x 1249	8 samples per pixel, black background

One fix was needed for InstantNGP. colmap2nerf.py puts the scene centre where the camera axes meet, but in a sideways capture the axes are nearly parallel, so this point ended up 3.4 units behind the wall and the shelf was far from the centre of the volume. The poses were recentred on the median of the COLMAP points and rescaled so that the farthest camera is 1 unit away. This only moves and scales the scene, so the images and relative poses stay the same.

3. Evaluation protocol

PSNR and SSIM measure how close the renders are to the ground truth pixel by pixel and in local structure, and LPIPS [4] measures how similar they look. The same script was used for both methods, and the official 3DGS metrics.py gave the same PSNR and SSIM (36.030 / 0.9789), which confirms it. All numbers are averages over the 8 test views.

Training time is the wall-clock time of the whole run. Render time is averaged over 20 passes through the 8 test views after a short warm-up. Model size is the size of the saved file, and GPU memory is the peak usage of the whole GPU during training. Everything ran on the same RTX 4060 Laptop GPU (8 GB) under WSL2.

4. Results

4.1 Summary

	InstantNGP 35k	**InstantNGP 100k**	**3DGS 30k**
PSNR (dB) ↑	32.92	33.05	**36.03**
SSIM ↑	0.9590	0.9598	**0.9789**
LPIPS-VGG ↓	0.1626	0.1618	**0.1469**
LPIPS-Alex ↓	0.0606	0.0595	**0.0376**
Training time (wall clock)	**5.0 min** (301 s)	13.9 min (834 s)	43.2 min (2594 s)
Render time, quality setting	155.8 ms (6.4 FPS, 8 spp)	155.8 ms (6.4 FPS, 8 spp)	**17.4 ms (57.6 FPS)**
Render time, 1 spp	20.3 ms (49.2 FPS)	20.3 ms (49.2 FPS)	–
Model file	**23.5 MB**	**23.4 MB**	271.8 MB
Trainable parameters	12.87 M	12.87 M	67.79 M (1,148,987 Gaussians x 59)
Peak GPU memory during training	–	1209 MiB	4305 MiB

spp means samples per pixel, i.e. how many rays InstantNGP averages for each pixel. The quality numbers were measured at 8 spp, so the 8 spp time is the fair one to compare with 3DGS.

4.2 Per-view quality

View	PSNR NGP	PSNR 3DGS	Δ PSNR	SSIM NGP	SSIM 3DGS	LPIPS-VGG NGP	LPIPS-VGG 3DGS
0000	33.15	36.03	+2.88	0.9557	0.9783	0.2036	0.1759
0008	33.73	37.31	+3.58	0.9653	0.9826	0.1593	0.1435
0016	33.26	36.53	+3.27	0.9624	0.9821	0.1624	0.1434
0024	29.21	31.15	+1.93	0.9343	0.9605	0.1823	0.1634
0032	34.08	37.55	+3.46	0.9678	0.9840	0.1482	0.1470
0040	32.74	35.52	+2.77	0.9622	0.9796	0.1404	0.1327
0048	33.38	36.11	+2.73	0.9642	0.9809	0.1475	0.1330
0056	34.84	38.05	+3.21	0.9669	0.9829	0.1506	0.1366
Mean	**33.05**	**36.03**	**+2.98**	**0.9598**	**0.9789**	**0.1618**	**0.1469**

NGP = InstantNGP at 100k steps; 3DGS at 30k iterations; Δ = 3DGS − NGP.

3DGS wins on every view and every metric, by 1.93 to 3.58 dB in PSNR. Both methods do worst on view 0024, which is the test view farthest from a training camera (see Section 6).

4.3 Convergence and training time

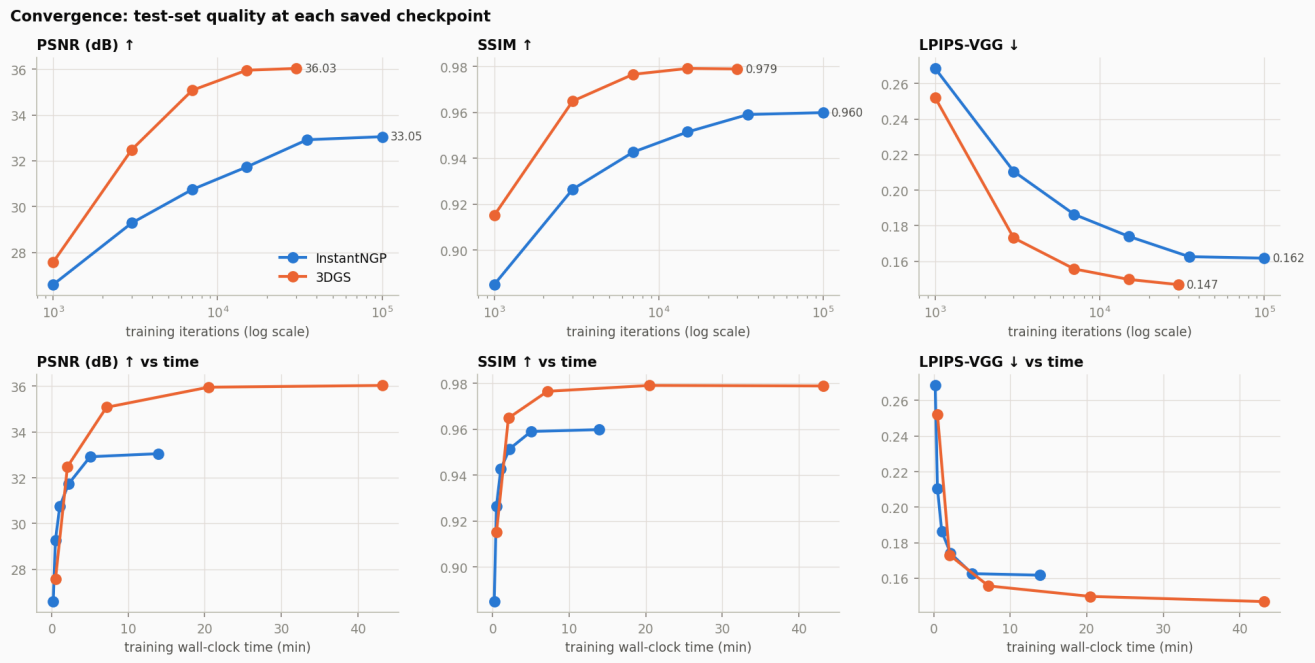


Figure 2. Test metrics against iterations (top, log scale) and against wall-clock training time (bottom).

Both methods had converged by the end. 3DGS improved by only 0.08 dB between 15k and 30k iterations, and InstantNGP by 0.13 dB between 35k and 100k steps, even though that took 2.8x longer. The table shows how long each method took to reach a given PSNR. The times are read off between checkpoints, so they are approximate.

Test PSNR reached	InstantNGP	3DGS
28 dB	**18 s**	33 s
30 dB	**42 s**	59 s
32 dB	2.6 min	**107 s**
33 dB	9.5 min	**2.6 min**
35 dB	not reached	6.9 min
Best reached	33.05 dB at 13.9 min	36.03 dB at 43.2 min

InstantNGP is quicker at the start, but only up to about 31 dB (around 1.3 min). After 7 minutes 3DGS is at 35.1 dB, which is already better than anything InstantNGP reached. The second half of 3DGS training adds just 0.08 dB. Densification stops at 15k, so the number of Gaussians stays fixed after that, but by then there are so many of them that each iteration takes about 91 ms. 3DGS also does much better on its training views than on the test views (40.30 dB vs 36.06 dB in its log at 30k).

4.4 Rendering time and model size

At the quality setting, 3DGS renders 9x faster (17.4 ms vs 155.8 ms per frame). InstantNGP at 1 spp is almost as fast (20.3 ms), but the images are noticeably noisier. 3DGS rasterises its 1.15 M Gaussians: they are projected to 2D, depth-sorted per tile and alpha-blended in a single pass. InstantNGP instead ray-marches through its occupancy grid and evaluates the hash encoding and both MLPs at every sample along the ray, and at 8 spp this is repeated eight times per pixel.

The model size of 3DGS increases during training, because densification keeps adding Gaussians until 15k iterations. The final file is 271.8 MB, 11.6x bigger than InstantNGP's 23.4 MB. Each of the 1,148,987 Gaussians stores 62 floats, and 48 of them are spherical-harmonic colour coefficients. The InstantNGP model (12.87 M fp16 parameters, mostly hash table entries) has a fixed size from the first step.

4.5 Visual comparison

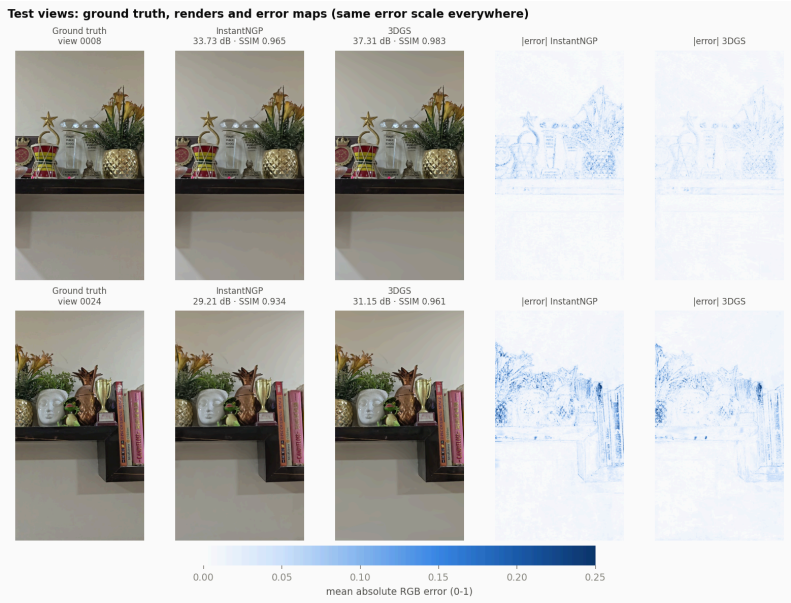


Figure 3. Ground truth, InstantNGP, 3DGS and absolute error maps (common scale) for the view with the largest PSNR gap (0008) and the worst view (0024).

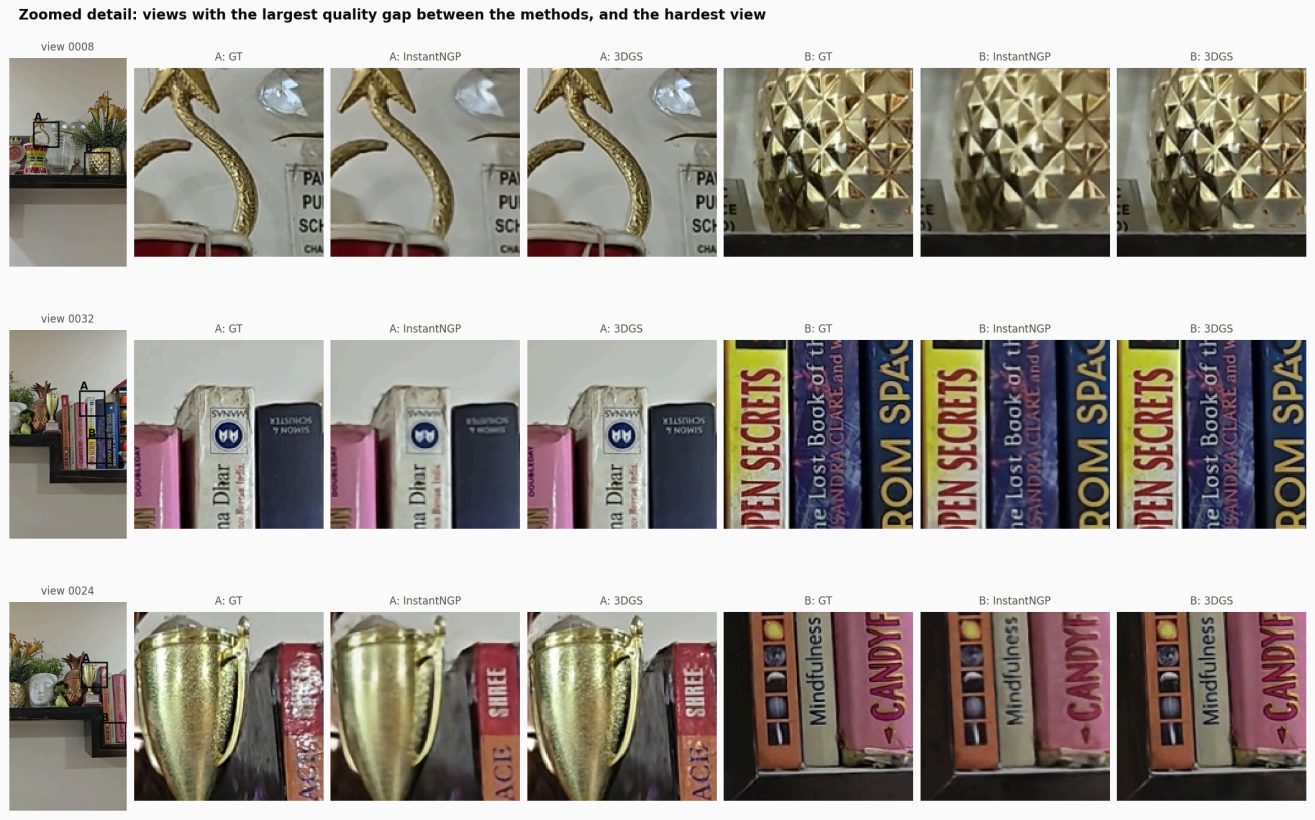


Figure 4. Crops at the highest-detail regions of the views with the largest PSNR gap (0008, 0032) and of the hardest view (0024).

Both methods get the overall shape, the colours and the shadows under the shelf right, and there are no floaters in the test views. Most of the error sits on edges and textured objects, and the plain wall is almost error-free for both.

The difference shows up in the fine details. In 3DGS the small text on the book spines ("Simon & Schuster", barcodes) and the facets of the gold pot look very close to the ground truth. In InstantNGP they are blurry and the small text cannot be read (Figure 4, rows 1–2). The trophy in view 0024 has a fine sparkly texture; 3DGS keeps most of it, but InstantNGP turns it into a smooth highlight. The sharpness numbers agree with this: measured by the variance of the Laplacian, InstantNGP renders have 53 % of the ground-truth sharpness on average and 3DGS renders 90 %.

4.6 Novel views away from the capture path

Seven extra camera paths were rendered with the same cameras for both methods: the original path (smoothed), the path moved up and down, a spiral, a dolly, a ±30° orbit and a ±20° look-around.

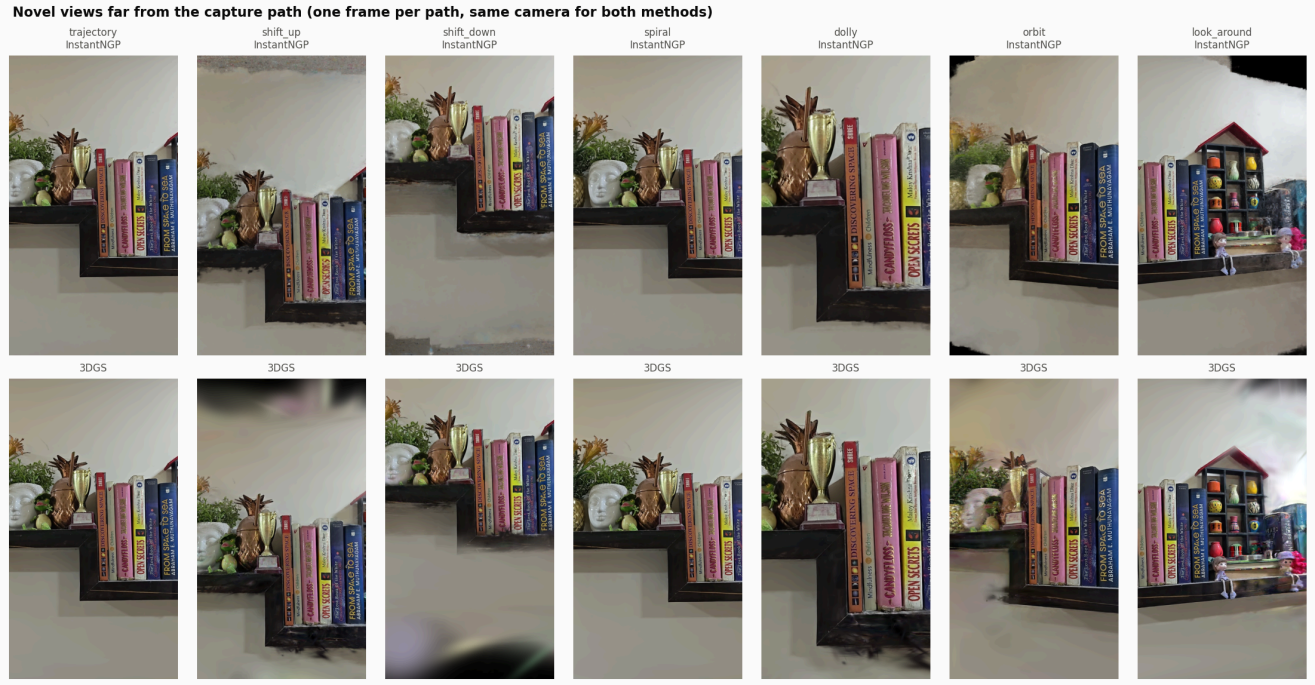


Figure 5. One frame per path at its largest deviation from the captured views; top InstantNGP, bottom 3DGS, same camera.

Near the original path (trajectory, spiral, dolly) both look clean and 3DGS is sharper, as on the test views. Moving the camera up or down shows parts of the wall that were never filmed. InstantNGP fills them with grainy noise and 3DGS with smooth dark or coloured blobs, which come from large Gaussians that nothing constrained. In the orbit and look-around paths, InstantNGP goes black where rays leave its bounding box, and 3DGS smears splats along the side of the shelf. The shelf itself is still easy to recognise in all of them. Since the video was one horizontal pass, neither method can show things that were hidden from that line, like the tops of the books.

5. Conclusion

On this scene 3DGS gives clearly better quality (+2.98 dB PSNR, better on all 8 views) and renders 9x faster at that quality. InstantNGP trains 8.6x faster with its default schedule, its model is 11.6x smaller and it uses less GPU memory (1.2 GB vs 4.3 GB).

Most of this comes down to how each method stores the scene. 3DGS ended up with 1.15 M Gaussians, and densification put them where the detail is (text, leaves, the faceted pot). That is why its renders are sharp, and also why its file is large. InstantNGP keeps a fixed-size hash grid and two small MLPs. It is compact and quick to train, but fine detail gets blurred. The speed difference has the same cause: drawing Gaussians takes one pass per image, but volume rendering needs many network queries per ray.

Neither method holds up well far from the captured path. They fail in different ways, with noise and black areas in InstantNGP and smeared splats in 3DGS.

6. Key observations

1. The shiny objects were the hardest part of the scene. Both methods had their largest errors on the trophy and the gold pot, where the highlights change as the camera moves.
2. Quality depended a lot on how close a test view was to the training cameras. The walking speed was not even, so views 0040 and 0024 ended up 1.7 and 2.1 camera spacings from the nearest training camera, while the rest were within 0.7. Those two were the worst views for both methods.

3. 3DGS did not need its full 30k iterations here. It was within 0.08 dB of its final PSNR at 15k, so training could have stopped there in about half the time.

7. Links

- Code, data, results and setup instructions: github.com/Soham-Gaonkar/Comparing-InstantNGP-and-3DGS
- Interactive 3DGS model in the browser: antimatter15 splat viewer

References

[1] T. Müller, A. Evans, C. Schied, A. Keller. Instant Neural Graphics Primitives with a Multiresolution Hash Encoding. ACM Transactions on Graphics (SIGGRAPH), 2022.

[2] B. Kerbl, G. Kopanas, T. Leimkühler, G. Drettakis. 3D Gaussian Splatting for Real-Time Radiance Field Rendering. ACM Transactions on Graphics (SIGGRAPH), 2023.

[3] J. L. Schönberger, J.-M. Frahm. Structure-from-Motion Revisited. CVPR, 2016.

[4] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, O. Wang. The Unreasonable Effectiveness of Deep Features as a Perceptual Metric (LPIPS). CVPR, 2018.